

Étude numérique des projections L^2 et H^1 dans des espaces de neurones finis ReLU^k

Yingxi Ye
Rapport de recherche

23 juin 2026

Résumé

Ce rapport présente une étude numérique des projections L^2 et H^1 dans des espaces de neurones finis construits à partir de fonctions ReLU^k . L'idée principale consiste à fixer les paramètres internes du réseau et à n'optimiser que les coefficients linéaires extérieurs, ce qui transforme le problème non convexe d'entraînement en un problème linéaire de projection ou de moindres carrés.

L'objectif est d'étudier à la fois la décroissance de l'erreur lorsque le nombre de neurones augmente et les contraintes numériques qui apparaissent en pratique : choix de la méthode d'intégration, nombre de points nécessaires, conditionnement des matrices et coût des méthodes de référence. Les expériences en dimensions deux et trois montrent que les espaces linéarisés ReLU^k donnent des approximations précises, mais que la performance dépend fortement de l'équilibre entre erreur d'approximation, erreur d'intégration et stabilité numérique.

1 Motivation et contexte

Les réseaux de neurones sont aujourd'hui très utilisés pour l'approximation de fonctions et la résolution numérique d'équations aux dérivées partielles. Dans les méthodes classiques de type PINN ou Deep Ritz, on cherche généralement à optimiser à la fois les paramètres internes et externes du réseau. Cette optimisation est non convexe et peut être difficile à contrôler numériquement.

Une alternative consiste à considérer des réseaux de neurones linéarisés. Les paramètres internes sont fixés à l'avance, tandis que seuls les coefficients linéaires de sortie sont calculés. Cette approche est proche des random feature methods et des extreme learning machines. Elle présente cependant une structure mathématique plus simple : l'espace d'approximation devient un espace vectoriel de dimension finie, et le calcul des coefficients se ramène à un problème de moindres carrés.

Ce cadre est particulièrement intéressant pour l'analyse numérique. Il fait apparaître des questions proches de celles rencontrées dans les méthodes d'éléments finis, les méthodes spectrales ou les méthodes de collocation : construction de bases, intégration numérique, assemblage de matrices, stabilité des solveurs et complexité algorithmique.

2 Cadre théorique

2.1 Espace de neurones finis ReLU^k

On considère le domaine

$$\Omega = [-1, 1]^d, \quad d = 2 \text{ ou } d = 3,$$

et l'activation

$$\sigma_k(t) = (t)_+^k = \max(t, 0)^k.$$

Pour des paramètres fixés $(w_j, b_j) \in \mathbb{R}^d \times \mathbb{R}$, on définit les fonctions de base

$$\phi_j(x) = \sigma_k(w_j \cdot x + b_j), \quad j = 1, \dots, n.$$

L'espace de neurones finis est alors

$$L_n^k = \text{span}\{\phi_1, \dots, \phi_n\}.$$

L'approximation cherchée s'écrit

$$u_n(x) = \sum_{j=1}^n a_j \phi_j(x),$$

où les coefficients a_j sont les seules inconnues.

Par homogénéité de ReLU^k , il est naturel de normaliser les paramètres en posant

$$\theta_j = (w_j, b_j) \in \mathbb{S}^d \subset \mathbb{R}^{d+1}, \quad \tilde{x} = (x, 1).$$

On peut alors écrire

$$\phi_j(x) = \sigma_k(\theta_j \cdot \tilde{x}).$$

Le choix des paramètres θ_j est important. Lorsque ces points sont bien distribués sur la sphère, l'espace L_n^k possède de bonnes propriétés d'approximation dans des espaces de Sobolev. Le cadre étudié ici est donc intermédiaire entre les méthodes classiques d'approximation linéaire et les méthodes modernes de réseaux de neurones.

2.2 Projection L^2

La projection L^2 de u sur L_n^k est définie par

$$u_n = \arg \min_{v \in L_n^k} \|u - v\|_{L^2(\Omega)}.$$

En écrivant $u_n = \sum_{j=1}^n a_j \phi_j$, on obtient formellement le système linéaire

$$Ma = f,$$

où

$$M_{ij} = \int_{\Omega} \phi_i(x) \phi_j(x), dx, \quad f_i = \int_{\Omega} u(x) \phi_i(x), dx.$$

La matrice M est une matrice de Gram, analogue à une matrice de masse dans une méthode de Galerkin.

2.3 Projection H^1

Pour la projection H^1 , on minimise

$$\|u - u_n\|_{H^1(\Omega)}^2 = \|u - u_n\|_{L^2(\Omega)}^2 + \|\nabla u - \nabla u_n\|_{L^2(\Omega)}^2.$$

La dérivée d'une fonction de base est

$$\nabla \phi_j(x) = k(w_j \cdot x + b_j)_+^{k-1} w_j.$$

La projection H^1 fait donc intervenir à la fois la matrice de masse et une matrice de rigidité :

$$S_{ij} = \int_{\Omega} \nabla \phi_i(x) \cdot \nabla \phi_j(x), dx.$$

La difficulté numérique est plus forte que dans le cas L^2 , car les dérivées rendent les erreurs d'intégration et les effets de conditionnement plus visibles.

2.4 Taux théoriques attendus

Dans le cas ReLU^k , la théorie des espaces de neurones finis linéarisés suggère que, pour des paramètres quasi-uniformes sur la sphère, on peut obtenir des taux de la forme

$$\inf_{v \in L_n^k} \|u - v\|_{H^m(\Omega)} \lesssim n^{-\frac{1}{2} - \frac{2(k-m)+1}{2d}}.$$

Ainsi, pour $d = 2$ et $k = 3$, on attend approximativement

$$\varepsilon_n^{L^2} \sim n^{-9/4}, \quad \varepsilon_n^{H^1} \sim n^{-7/4}.$$

Ces taux servent de référence pour interpréter les expériences. Toutefois, les courbes numériques ne dépendent pas uniquement de l'erreur d'approximation théorique : elles dépendent aussi de la qualité de l'intégration numérique, du solveur linéaire et du conditionnement des systèmes discrets.

3 Formulation numérique

3.1 Moindres carrés pondérés

Une manière naturelle de calculer la projection consiste à assembler les matrices M et S . Cependant, former explicitement les équations normales peut être instable. Si l'on note Φ la matrice des valeurs des fonctions de base sur des points d'intégration, alors

$$M \approx \Phi^\top W \Phi.$$

Or le passage aux équations normales amplifie le conditionnement :

$$\kappa(\Phi^\top W \Phi) \simeq \kappa(\sqrt{W} \Phi)^2.$$

Pour cette raison, on résout plutôt le problème de moindres carrés pondérés

$$\min_a \|Aa - y\|_2,$$

avec

$$A_{qj} = \sqrt{\omega_q} \phi_j(x_q), \quad y_q = \sqrt{\omega_q} u(x_q).$$

Dans le cas H^1 , on ajoute les lignes correspondant aux dérivées :

$$A = \begin{bmatrix} \sqrt{W} \Phi \\ \sqrt{W} \partial_{x_1} \Phi \\ \dots \\ \sqrt{W} \partial_{x_d} \Phi \end{bmatrix}, \quad y = \begin{bmatrix} \sqrt{W} u \\ \sqrt{W} \partial_{x_1} u \\ \dots \\ \sqrt{W} \partial_{x_d} u \end{bmatrix}.$$

Cette formulation permet d'utiliser des solveurs de moindres carrés robustes, notamment basés sur la SVD.

3.2 Méthodes d'intégration comparées

On compare plusieurs stratégies pour générer les points et poids (x_q, ω_q) .

QUAD-box. Le domaine est découpé en petits carrés en dimension deux ou en petits cubes en dimension trois. Sur chaque sous-domaine, on applique une quadrature de Gauss-Legendre tensorielle. Cette méthode est simple, déterministe et évite de construire explicitement les régions d'activation des ReLU.

Monte Carlo et Sobol-QMC. La méthode Monte Carlo utilise des points aléatoires uniformes. La méthode Sobol-QMC utilise des points de faible discrédance. Ces méthodes sont simples à étendre en dimension plus élevée et permettent de construire rapidement des problèmes de moindres carrés pondérés.

POLY et QUAD-simplex. Pour deux fonctions de base,

$$\phi_i(x) = (w_i \cdot x + b_i)_+^k, \quad \phi_j(x) = (w_j \cdot x + b_j)_+^k,$$

le produit $\phi_i \phi_j$ est non nul seulement sur la région effective

$$D_{ij} = \Omega \cap \{w_i \cdot x + b_i > 0\} \cap \{w_j \cdot x + b_j > 0\}.$$

En dimension deux, D_{ij} est un polygone convexe ; en dimension trois, c'est un polyèdre convexe. Les méthodes POLY et QUAD-simplex exploitent cette géométrie. Elles sont très précises, mais leur coût augmente rapidement car une construction géométrique est nécessaire pour chaque paire (i, j) .

Schéma méthodologique de l'étude numérique

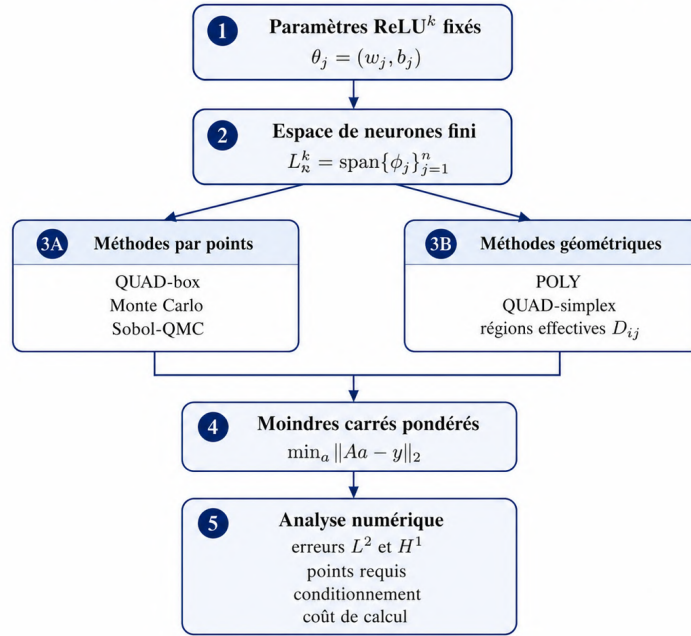


FIGURE 1 – Schéma méthodologique de l'étude numérique. Les paramètres internes des neurones sont fixés, puis les coefficients linéaires sont calculés par moindres carrés pondérés.

4 Expériences numériques

4.1 Fonctions cibles lisses

On commence par des fonctions cibles lisses. En dimension deux :

$$u(x, y) = \sin\left(\frac{\pi x}{2}\right) \sin\left(\frac{\pi y}{2}\right).$$

En dimension trois :

$$u(x, y, z) = \sin\left(\frac{\pi x}{2}\right) \sin\left(\frac{\pi y}{2}\right) \sin\left(\frac{\pi z}{2}\right).$$

Ces fonctions permettent d'étudier le comportement de base de l'espace L_n^k lorsque le nombre de neurones augmente.

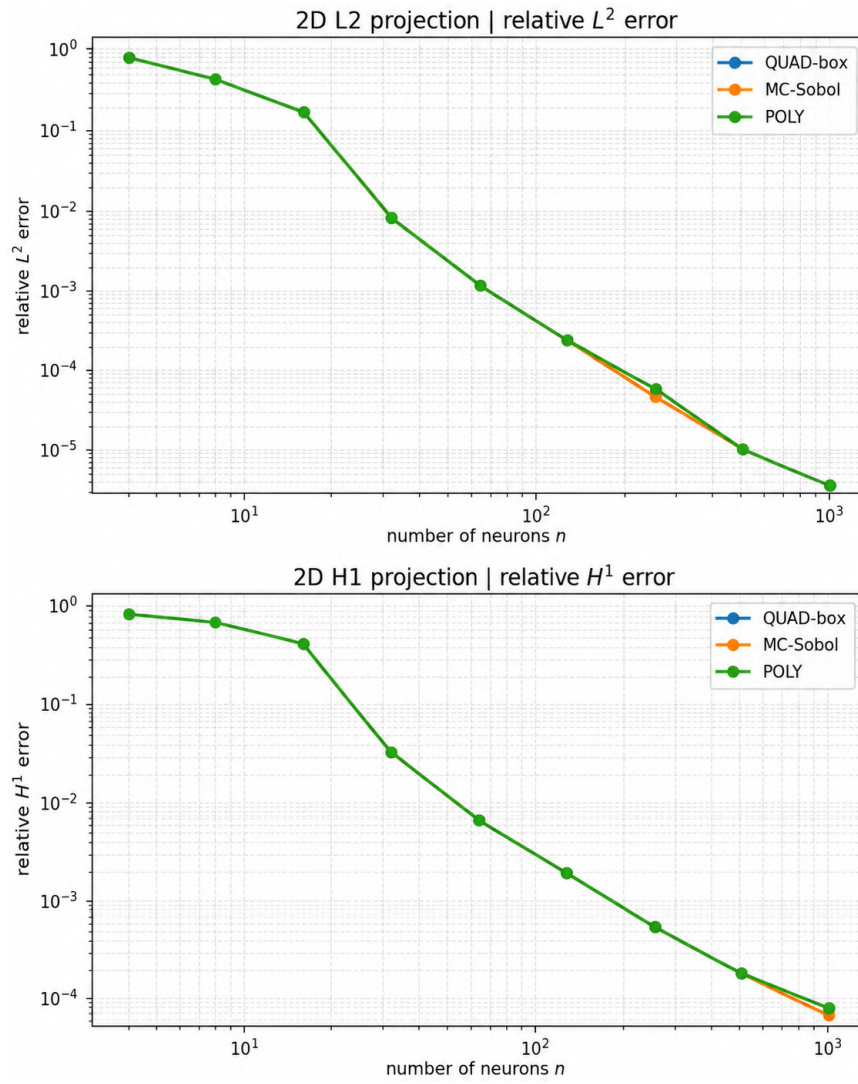


FIGURE 2 – Erreurs relatives des projections L^2 et H^1 en dimension deux pour une fonction cible lisse.

La Figure 2 montre une décroissance régulière des erreurs relatives lorsque n augmente. Les courbes obtenues par QUAD-box, MC-Sobol et POLY sont presque superposées, ce qui indique que l'erreur dominante vient principalement de l'approximation dans l'espace L_n^k , plutôt que de l'intégration numérique.

La proximité entre QUAD-box et POLY montre aussi que la quadrature sur boîtes fournit déjà une approximation fiable de la projection continue. La norme H^1 reste néanmoins plus exigeante, car elle fait intervenir les dérivées et devient donc plus sensible aux erreurs locales.

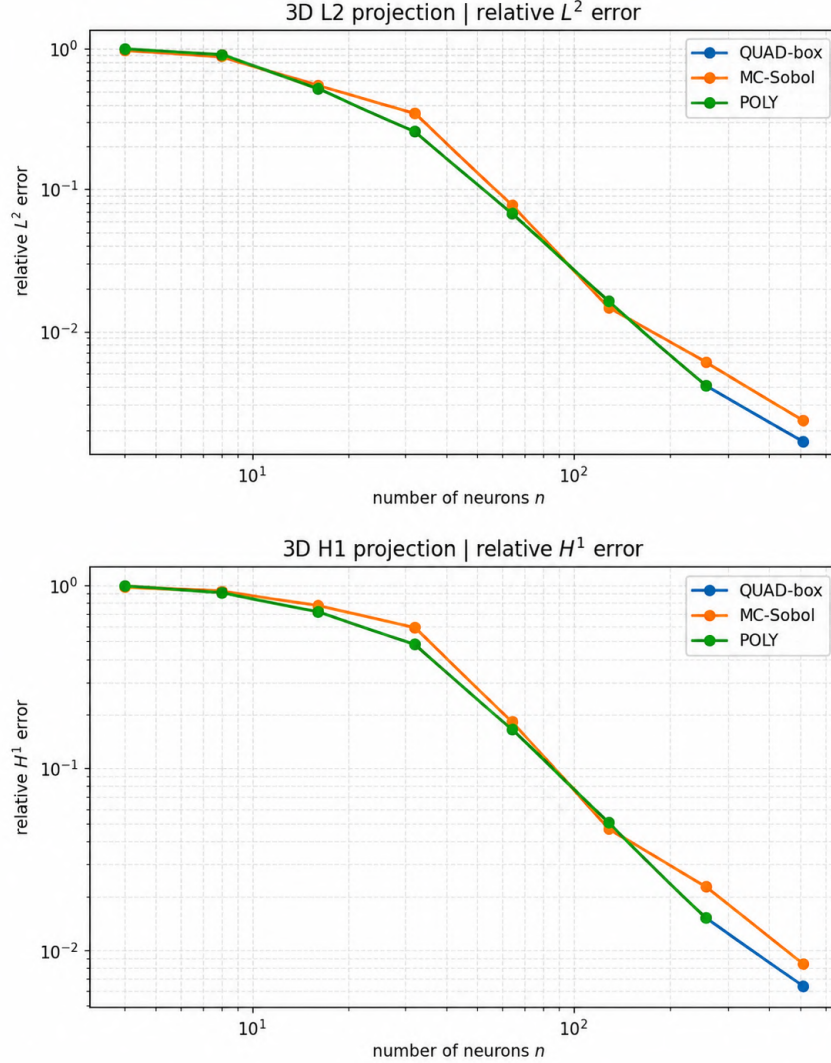


FIGURE 3 – Erreurs relatives des projections L^2 et H^1 en dimension trois pour une fonction cible lisse.

La Figure 3 confirme que la méthode reste efficace en dimension trois. Même si les intégrales sont plus coûteuses et que la construction géométrique des régions effectives devient plus complexe, les erreurs diminuent clairement avec le nombre de neurones. Les trois méthodes donnent des tendances cohérentes, ce qui montre que l'approche par moindres carrés pondérés conserve une bonne extensibilité au-delà de la dimension deux.

4.2 Nombre de points nécessaires

Une question centrale est la suivante : lorsque l'erreur d'approximation diminue avec n , combien de points d'intégration faut-il utiliser pour que l'erreur d'intégration ne devienne pas dominante ?

On utilise POLY comme référence déterministe. Pour chaque valeur de n , on cherche le plus

petit nombre de points tel que l'erreur obtenue par MC, Sobol ou QUAD-box soit comparable à l'erreur de référence. Ce nombre est appelé nombre de points requis.

Pour Monte Carlo, l'erreur d'intégration est typiquement de l'ordre

$$O(N^{-1/2}).$$

Pour Sobol-QMC, on espère un comportement plus favorable, proche de

$$O(N^{-1}),$$

à des facteurs logarithmiques près. En combinant ces estimations avec les taux d'approximation attendus pour $d = 2$ et $k = 3$, on obtient les lois heuristiques suivantes :

$$N_{MC,L^2} \sim n^{9/2}, \quad N_{MC,H^1} \sim n^{7/2},$$

et

$$N_{QMC,L^2} \sim n^{9/4}, \quad N_{QMC,H^1} \sim n^{7/4}.$$

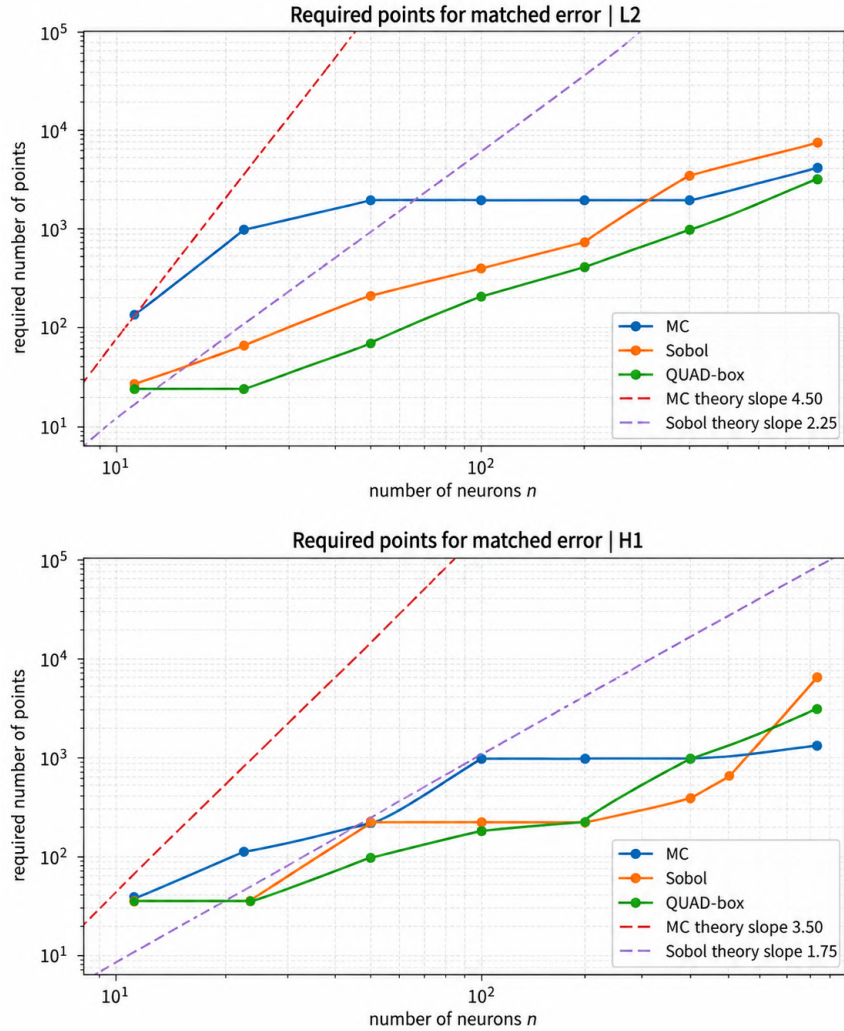


FIGURE 4 – Nombre de points requis pour obtenir une erreur comparable à la référence POLY. Les lignes pointillées indiquent les pentes théoriques attendues pour Monte Carlo et Sobol-QMC.

La Figure 4 met en évidence un phénomène important : lorsque n augmente, le nombre de points nécessaires augmente également. Ce comportement est naturel. Plus l'espace de neurones est

riche, plus l'erreur d'approximation théorique devient petite ; il faut donc réduire simultanément l'erreur d'intégration pour que la projection discrète reste proche de la projection continue.

Les courbes ne suivent pas parfaitement les pentes théoriques. Cela ne contredit pas nécessairement la théorie : l'expérience ne mesure pas une erreur d'intégration abstraite isolée, mais une erreur finale après résolution d'un problème de moindres carrés. Cette erreur dépend aussi du conditionnement de la matrice A , de la grille de valeurs candidates pour N , et du critère pratique utilisé pour décider si l'erreur est suffisamment proche de la référence. En particulier, lorsque n devient grand, une petite perturbation dans les produits scalaires peut être amplifiée par la mauvaise condition numérique du système.

4.3 Fonction cible de faible régularité

Pour tester la robustesse de la méthode, on considère ensuite une fonction moins régulière :

$$u(x, y) = \left| x - \frac{1}{2} \right|^{k+\frac{3}{2}} + \left| y - \frac{1}{2} \right|^{k+\frac{3}{2}}.$$

Cette fonction n'est pas aussi lisse que la fonction sinus précédente. Elle présente une régularité limitée près des droites $x = 1/2$ et $y = 1/2$.

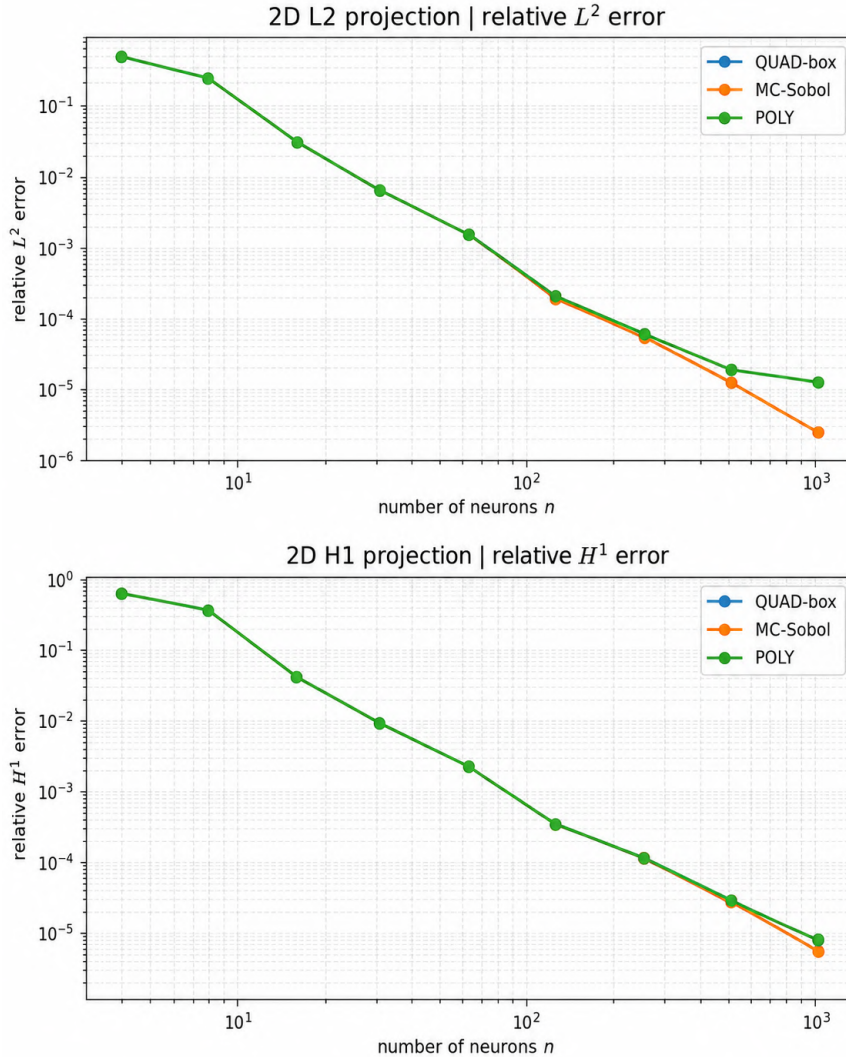


FIGURE 5 – Erreurs relatives des projections L^2 et H^1 en dimension deux pour une fonction cible de plus faible régularité.

La Figure 5 montre que les erreurs L^2 et H^1 diminuent encore clairement avec n . Le résultat est important car il montre que le bon comportement observé sur la fonction sinus n'est pas uniquement dû à une régularité excessive de la cible. L'espace L_n^k reste capable d'approximer des fonctions présentant des singularités faibles ou une régularité locale plus limitée.

On remarque cependant que la fin de la courbe peut être moins régulière. Cela peut être lié à deux effets. D'une part, la fonction cible étant moins régulière, le taux asymptotique peut être plus difficile à observer sur une plage finie de valeurs de n . D'autre part, lorsque l'erreur devient très petite, les effets d'intégration et de conditionnement deviennent plus visibles.

4.4 Analyse du coût et de la stabilité numérique

La dernière expérience compare les temps de calcul en dimension trois pour la projection H^1 . On sépare plusieurs composantes : génération des points, assemblage, résolution du système de moindres carrés et temps total.

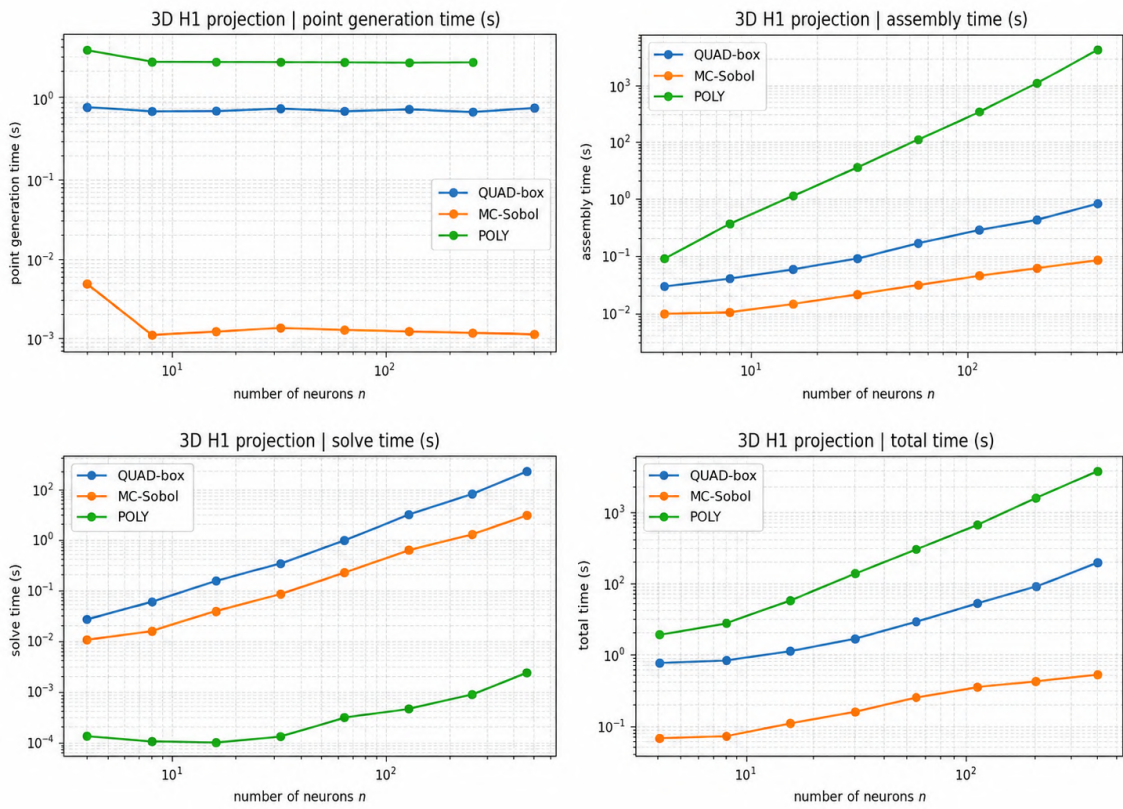


FIGURE 6 – Décomposition du temps de calcul et comparaison des méthodes pour la projection H^1 en dimension trois.

La Figure 6 montre que MC-Sobol est la méthode la plus rapide en temps total. Cela s'explique par le fait que les points sont générés directement dans le domaine et qu'aucune construction géométrique des régions d'activation n'est nécessaire. QUAD-box est plus coûteuse, mais reste relativement stable et prévisible : elle construit des points de Gauss sur des petits cubes réguliers, puis résout le problème de moindres carrés associé.

La méthode POLY est beaucoup plus coûteuse, surtout dans l'assemblage et le temps total. La raison est structurelle : pour chaque paire de fonctions de base, il faut construire la région effective D_{ij} . En dimension trois, cette région est un polyèdre convexe. Sa construction nécessite des opérations géométriques plus lourdes, comme la recherche de sommets, la récupération de la frontière par ConvexHull et une décomposition en tétraèdres ou en sous-domaines simples. Le coût augmente donc très rapidement avec n .

Cette figure confirme que POLY est surtout utile comme méthode de référence déterministe, mais qu'elle n'est pas la meilleure option pour des calculs à plus grande échelle. En revanche, QUAD-box et Sobol-QMC sont plus adaptées à une mise en œuvre numérique efficace, notamment en dimension trois.

5 Discussion

Les expériences mettent en évidence trois points principaux.

Premièrement, les espaces de neurones finis ReLU^k possèdent bien une capacité d'approximation forte en normes L^2 et H^1 . Les erreurs diminuent régulièrement en dimensions deux et trois, pour des fonctions lisses comme pour des fonctions de régularité plus faible.

Deuxièmement, la qualité de l'intégration numérique devient essentielle lorsque n augmente. Pour de petits espaces, l'erreur d'approximation domine. Pour de plus grands espaces, l'erreur d'intégration doit être suffisamment petite pour ne pas limiter la précision finale. L'expérience sur les points requis montre explicitement ce couplage entre nombre de neurones et nombre de points.

Troisièmement, le conditionnement est une difficulté centrale. Même si l'espace d'approximation est théoriquement efficace, la résolution numérique peut devenir instable lorsque les matrices de Gram ou les matrices de moindres carrés sont mal conditionnées. C'est pourquoi il est préférable de résoudre directement les problèmes de moindres carrés pondérés, plutôt que de former les équations normales. À plus long terme, l'étude de préconditionneurs adaptés aux bases ReLU^k constitue une direction naturelle.

6 Lien avec la majeure ANEDP

Ce projet se situe à l'interface de plusieurs thèmes de la majeure ANEDP : analyse numérique, équations aux dérivées partielles, approximation et calcul scientifique. D'un côté, il repose sur des notions théoriques comme les espaces de Sobolev, les projections L^2 et H^1 , les formulations variationnelles et les taux d'approximation. De l'autre, il implique des questions très concrètes de mise en œuvre : choix des quadratures, construction des matrices, stabilité des solveurs, coût CPU et extension à la dimension trois.

Le projet ouvre également vers des applications liées aux EDP. Une suite naturelle serait d'utiliser ces espaces de neurones finis pour résoudre des problèmes elliptiques ou paraboliques, puis d'étudier des variantes respectant des contraintes physiques. Par exemple, pour les équations de Stokes ou de Navier–Stokes incompressibles, il serait intéressant de construire des espaces de neurones exactement sans divergence, afin d'encoder la contrainte $\nabla \cdot u = 0$ directement dans l'espace d'approximation.

7 Conclusion et perspectives

Ce travail a étudié les projections L^2 et H^1 dans des espaces de neurones finis ReLU^k avec paramètres internes fixés. Les expériences montrent que ces espaces fournissent une approximation efficace, mais que la précision finale dépend d'un équilibre entre trois facteurs : l'erreur d'approximation, l'erreur d'intégration et la stabilité numérique.

Les perspectives principales sont les suivantes :

- étudier plus systématiquement le conditionnement et le spectre singulier des matrices de moindres carrés ;
- relier les irrégularités des courbes de points requis au conditionnement numérique ;
- développer des stratégies de préconditionnement adaptées aux bases ReLU^k ;
- appliquer la méthode à des EDP elliptiques et paraboliques ;
- construire des espaces de neurones respectant exactement certaines contraintes physiques, comme la contrainte de divergence nulle.

Ainsi, ce projet constitue une première étape vers une compréhension plus fine des méthodes de neurones linéarisés pour l'approximation et la résolution numérique d'équations aux dérivées partielles.

Références

- [1] X. Liu, T. Mao and J. Xu. *Integral Representations of Sobolev Spaces via ReLU^k Activation Function and Optimal Error Estimates for Linearized Networks*. arXiv preprint, 2025.
- [2] T. Mao, J. Xu and X. Xu. *Solving High-Dimensional PDEs Using Linearized Neural Networks*. arXiv preprint, 2026.
- [3] X. Liu, T. Mao and J. Xu. *The Stability of Shallow Neural Networks on Spheres : A Sharp Spectral Analysis*. arXiv preprint, 2025.
- [4] J. He, X. Liu and Z. Tian. *Divergence-free Linearized Neural Networks : Integral Representation and Optimal Approximation Rates*. arXiv preprint, 2026.